

BlenderSceneAgent

A Lightweight Agent-Driven 3D Scene Generation System
with Geometry-Safe Layout and Multi-Agent Coordination

1

¹School of Computer Science, Fudan University
23300680108@m.fudan.edu.cn

June 10, 2026

Abstract

Generating realistic 3D indoor scenes from natural language descriptions is a fundamental challenge at the intersection of computer graphics and generative AI. We present **BlenderSceneAgent**, a lightweight multi-agent framework that converts free-text user prompts into physically plausible 3D scenes rendered in Blender. Our system decomposes the generation task across four specialized agents—Planner, Validator, Repair, and Visual Critic—and introduces a novel *6-stage geometry-safe layout pipeline* that guarantees floor contact, wall attachment, room-boundary clamping, and iterative collision resolution using axis-aligned bounding boxes (AABB). Unlike end-to-end text-to-3D approaches, BlenderSceneAgent emits a structured *scene graph* JSON as an intermediate representation, making the generation process interpretable, debuggable, and editable. The system supports 25 object types, 9 spatial relations, and 6 scene categories, all implemented in approximately 5,600 lines of Python with 81 unit tests that run without Blender. We demonstrate that a deterministic layout pipeline, combined with rule-based multi-agent orchestration, produces visually coherent scenes without GPU training or external 3D assets.

Code: https://github.com/cainiaopig/Graphics_pj3/tree/master

1 Introduction

The rapid advancement of large language models (LLMs) and vision-language models (VLMs) has opened new frontiers in computer graphics, particularly in automating content creation pipelines. Recent work has demonstrated that multi-agent systems can effectively tackle complex 2D generation tasks such as poster design (??), 3D asset modeling via Blender Python code (?), and large-scale embodied-AI scene population (?). These systems share a common insight: decomposing a creative task across specialized agents—each with distinct responsibilities—yields more reliable results than a single end-to-end model.

However, existing 3D scene generation systems face several challenges. Systems that rely on LLMs to directly write Blender Python code (?) risk introducing syntax errors, physically impossible placements, and geometries that penetrate walls or floors. Neural approaches based on NeRF or 3D Gaussian Splatting require substantial GPU resources and produce opaque representations that are difficult to edit. Large-scale frameworks like SAGE (?) depend on external 3D asset libraries and complex simulation environments, making them impractical for rapid prototyping and classroom use.

We propose **BlenderSceneAgent**, a lightweight alternative that addresses these limitations through three design principles:

1. **Scene Graph as Intermediate Representation (IR).** Rather than generating raw Blender code, our agents produce a structured JSON scene graph—a declarative description of objects, their spatial relations, and room configuration. This IR is human-readable, version-controllable, and serves as a safety barrier between the LLM’s probabilistic output and the deterministic Blender execution.
2. **Geometry-Safe Layout Pipeline.** All 3D coordinates are computed deterministically by a 6-stage pipeline that enforces physical constraints: floor contact, wall attachment with thickness offset, room

boundary clamping, and iterative AABB collision resolution. The LLM is *never* asked to produce raw coordinates.

3. **Multi-Agent Orchestration with Mock-First Design.** The system operates in two modes: a *mock* mode using keyword matching and rule-based heuristics (requiring no API keys), and an *LLM-driven* mode that connects to any OpenAI-compatible API. This design enables rapid iteration, testing, and deployment without external dependencies.

Our contributions are:

- A complete end-to-end pipeline from natural language to rendered 3D scene, implemented in ~5,600 lines of Python.
- A novel 6-stage geometry-safe layout algorithm with AABB collision detection and resolution.
- Support for 25 object types, 9 spatial relations, and 6 scene categories using only Blender primitives (no external assets).
- 81 pure-Python unit tests validating the entire non-Blender pipeline.
- Open-source release under a permissive license.

2 Related Work

2.1 Agent-Driven 2D Generation

GenPilot (?) introduced a multi-agent test-time prompt optimization system for text-to-image generation, decomposing the problem into error analysis and prompt refinement stages with VQA-based evaluation. Paper2Poster (?) automated the conversion of academic papers into conference posters using a Parser–Planner–Painter architecture with iterative visual feedback. PosterForest (?) extended this concept with hierarchical multi-agent collaboration, explicitly modeling document structure as a tree that is jointly optimized with the poster layout. These systems demonstrate the effectiveness of the Plan–Validate–Repair agent pattern, which BlenderSceneAgent adopts and extends into the 3D domain.

2.2 3D Scene and Asset Generation

SAGE (?) is a large-scale agentic 3D scene generation framework targeting embodied AI simulation. It uses LLM agents to populate rooms with objects from a 565K-asset library, producing physically stable scenes for robot training. However, SAGE requires significant infrastructure (simulators, asset databases) and is not designed for lightweight interactive use.

LL3M (?) reframes 3D modeling as a code-generation task: LLM agents write Blender Python scripts to create and edit 3D assets. While powerful, this approach gives the LLM direct control over geometry, which can lead to errors and physically invalid placements. BlenderSceneAgent adopts the multi-agent philosophy of LL3M but constrains the LLM to a declarative scene graph, reserving geometric computation for a deterministic layout engine.

CAD-Assistant (?) demonstrated tool-augmented VLM frameworks for CAD tasks, using FreeCAD’s Python API as an execution environment. This work validates the broader paradigm of AI-as-planner with domain-specific tools as executor.

WorldCraft (?) enables photo-realistic 3D world creation through LLM agents, focusing on outdoor scenes and terrain generation. Idea23D (?) generates 3D models from interleaved multimodal inputs using collaborative LLM agents. These systems, together with comprehensive surveys (?), highlight the growing interest in agent-driven 3D content creation.

2.3 Blender-Based Pipeline Design

Blender’s Python API (`bpy`) has become a popular backend for procedural 3D content generation. The key design challenge is *module isolation*: the `bpy` module is only available inside Blender’s embedded Python interpreter, so pipeline code must be carefully separated into Blender-independent (planning, validation) and Blender-dependent (geometry creation, rendering) components. `BlenderSceneAgent` addresses this through strict module boundaries—all code under `blender/` lazily imports `bpy`, and the main pipeline (`core/pipeline.py`) operates without Blender.

3 Method

3.1 System Architecture

The system follows a 5-stage pipeline (Figure ??):

1. **Plan:** The Planner Agent converts a natural language prompt into a **SceneGraph**—a Pydantic-validated JSON object containing room dimensions, object instances (each with type, size, semantic position, and material), spatial relations, lighting configuration, and camera parameters.
2. **Validate:** The Validator Agent runs 9 rule-based checks: missing object detection, unsupported type detection, invalid relation detection, duplicate ID detection, illegal dimension warnings, style coverage check, and room-surface relation legality.
3. **Repair:** If validation fails, the Repair Agent applies 5 categories of rule-based fixes: removing unsupported types, fixing invalid relation targets, correcting room-surface relations, deduplicating IDs, and clamping illegal dimensions.
4. **Build & Render:** The Repaired Scene Graph is passed to Blender. The `scene_builder` module constructs the room shell, runs the 6-stage geometry-safe layout pipeline (Section ??), creates each object using Blender primitives, configures materials and lighting, and renders the final image.
5. **Critique (future):** The Visual Critic Agent (placeholder in current version) will accept the rendered image and evaluate it against the original prompt using a VLM.

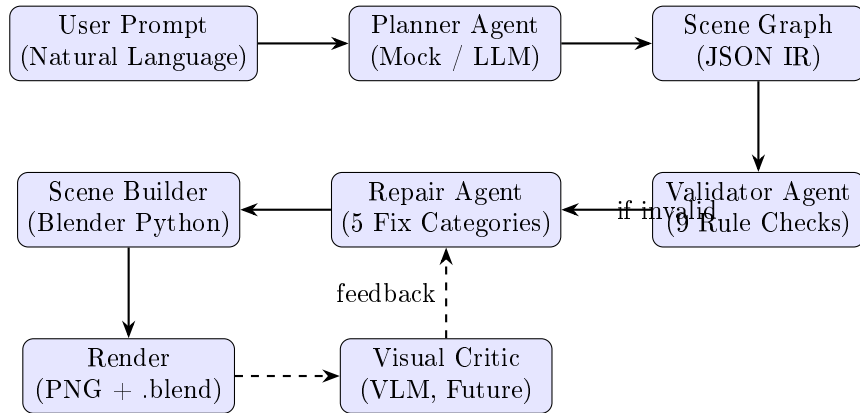


Figure 1: System architecture of `BlenderSceneAgent`. Solid arrows denote the main pipeline; dashed arrows denote the planned visual critique feedback loop.

3.2 Scene Graph Intermediate Representation

The Scene Graph is the central data structure that decouples AI planning from Blender execution. It is defined using Pydantic v2 with strict type validation:

- **SceneGraph**: top-level container with `scene_id`, `scene_type`, `style`, `room`, `objects`, `relations`, `lighting`, `camera`, and `metadata`.
- **SceneObject**: each object has `id`, `type`, `description`, `semantic_position` (e.g., “left wall”, “center”), `size` (meters in $[x, y, z]$), `material`, `color`, `rotation`, and `extra` properties.
- **SpatialRelation**: a directed triple (`subject`, `relation`, `target`) where `relation` $\in$ {on, near, left_of, right_of, in_front_of, behind, attached_to, inside, facing}.

This design ensures that the LLM never emits raw 3D coordinates—it only describes *what* objects exist and *how* they relate to each other. The actual Blender positions are computed by the layout pipeline (Section ??).

3.3 6-Stage Geometry-Safe Layout Pipeline

The layout pipeline is the core technical contribution of this work. Given a set of objects with sizes and a set of spatial relations, it deterministically computes every object’s 3D center position such that all physical constraints are satisfied. The pipeline operates in six sequential stages (Figure ??):

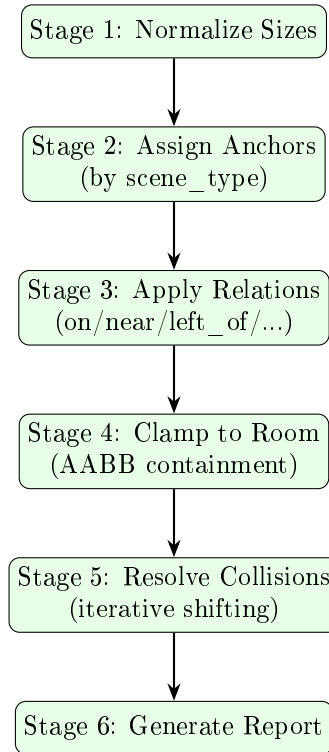


Figure 2: The 6-stage geometry-safe layout pipeline. All stages are pure Python (no Blender required), enabling fast testing and debugging.

Stage 1: Normalize Sizes

Every object is assigned a default size $[w, d, h]$ from a lookup table of 25 supported types (e.g., bed: $2.2 \times 1.4 \times 0.55$, desk: $1.6 \times 0.7 \times 0.75$). If the scene graph specifies a custom size, it overrides the default after validation. Each object is wrapped in an **AABB** dataclass with `center` and `size` vectors.

Stage 2: Assign Anchors by Scene Type

Objects are categorized as `floor`, `wall`, or `on_target`. Each scene type has a deterministic anchor function that maps objects to initial (x, y) positions:

- **Bedroom:** bed $\rightarrow$ left-back, desk $\rightarrow$ right-back, carpet $\rightarrow$ center, window $\rightarrow$ back wall, poster $\rightarrow$ left wall, neon_light $\rightarrow$ back wall upper.
- **Classroom:** blackboard $\rightarrow$ back wall center, desks $\rightarrow$ grid in room center, chairs $\rightarrow$ near corresponding desks.
- **Coffee Shop:** counter $\rightarrow$ back or side, tables $\rightarrow$ grid, menu_board $\rightarrow$ above counter.
- **Laboratory:** desks $\rightarrow$ left and right sides, glowing_tube $\rightarrow$ on desk, warning_sign $\rightarrow$ wall.
- **Generic:** fallback grid-based placement.

Floor objects are placed with `center_z = height/2` (sitting exactly on the floor). Wall objects receive a `center_z = room_height  $\times$  height_ratio`.

Stage 3: Apply Spatial Relations

Relations are processed in a predefined order (`attached_to` $\rightarrow$ `on` $\rightarrow$ `inside` $\rightarrow$ `near` $\rightarrow$ `left_of/right_of` $\rightarrow$ `in_front_of/behind`). Key behaviors:

- **on:** subject center = $(x_{\text{target}}, y_{\text{target}}, z_{\text{target}}^{\text{max}} + h_{\text{subject}}/2 + 0.01)$.
- **attached_to:** subject placed at wall surface with `WALL_THICKNESS` offset (0.08 m).
- **near:** subject shifted toward target with a 0.3 m padding gap.
- **left_of / right_of / in_front_of / behind:** subject shifted relative to target with padding.

Critically, all relation handlers preserve the subject’s original z -coordinate (floor vs. wall vs. on-target), preventing objects from floating or sinking.

Stage 4: Clamp to Room

Each AABB is checked against the room’s usable volume (room dimensions minus `MARGIN` of 0.08 m on each side). If an object protrudes beyond the boundary, its center is shifted so the AABB fits entirely inside. Wall objects that are intentionally placed outside the room interior (attached to walls with thickness offset) are exempt from out-of-bounds checking.

Stage 5: Resolve Collisions

Only floor-object pairs are considered (wall objects and `on` pairs are excluded). Objects are processed in priority order (bed first, then desk, table, etc.). For each intersecting pair, the later object is shifted along $\pm x$ then $\pm y$ in 0.3 m steps, up to 20 attempts. After each shift, the object is re-clamped to room bounds. Unresolved collisions are recorded as warnings.

The collision detection uses the standard AABB overlap test with configurable padding:

$$\text{overlap}(a, b, \text{padding}) \iff |a.\text{center}_x - b.\text{center}_x| < \frac{a.\text{size}_x + b.\text{size}_x}{2} + \text{padding} \wedge \dots \text{ (same for } y \text{ and } z)$$

Stage 6: Generate Report

The pipeline emits a `LayoutReport` containing: per-object positions and AABBs, collision pairs before and after resolution, out-of-bounds objects before and after clamping, and any warnings. This report is saved as JSON alongside the rendered image for debugging and analysis.

3.4 Backward Compatibility

The new pipeline maintains backward compatibility with the original `apply_relation_layout()` API through thin wrapper functions. Existing `resolve_semantic_position()` and `estimate_initial_location()` functions are preserved as internal helpers.

3.5 Blender Scene Construction

Room Shell

The room is built using the stage-set convention: floor, ceiling, back wall, left wall, and right wall—the front wall is omitted so the camera (placed outside the room) can see inside. Walls are positioned *outside* the usable volume, offset by `WALL_THICKNESS` (0.08 m):

- Floor: center (0, 0, $-\text{FLOOR_THICKNESS}/2$)
- Back wall: center (0, $\text{depth}/2 + \text{WALL_THICKNESS}/2$, $\text{height}/2$)
- Left wall: center ($-\text{width}/2 - \text{WALL_THICKNESS}/2$, 0, $\text{height}/2$)
- Right wall: center ($\text{width}/2 + \text{WALL_THICKNESS}/2$, 0, $\text{height}/2$)

Object Creation

All box-shaped objects are created using a `create_box()` helper that sets `obj.dimensions` directly and calls `bpy.ops.object.transform_apply(scale=True)`, ensuring consistent world-space sizing regardless of the underlying mesh. Cylindrical objects (furniture legs, tubes, plants) use `create_cylinder_box()` with the same pattern. This replaces the previous approach of setting `obj.scale`, which was fragile when objects had non-unit base meshes.

Materials and Lighting

Materials are assigned by name lookup from a library of diffuse, emissive, and glass shaders. Objects with emissive properties (`neon_light`, `glowing_tube`, monitor screens, lamps) automatically receive point lights matching their emission color when `use_emissive_lights` is enabled in the scene graph.

3.6 Implementation Details

The system is implemented in approximately 5,600 lines of Python across 29 files, organized into six packages:

- `agents/` (6 files): Agent base class and four agent implementations with LLM client stub.
- `blender/` (6 files): Blender-dependent modules for primitives, materials, layout, scene building, geometry validation, and render config.
- `core/` (4 files): Pipeline orchestration, JSON utilities, file utilities, and logging.
- `schema/` (2 files): Pydantic v2 data models for the scene graph.
- `scripts/` (3 files): Blender runtime script, batch demo, and turntable export stub.
- `tests/` (4 files): 81 unit tests covering schema, validator, and layout modules.

All tests run without Blender—the layout and validation logic is pure Python. This is a deliberate design choice that enables fast CI/CD iteration.

4 Experimental Results

4.1 Scene Generation

We evaluated the system on five scene types using mock-mode planning (rule-based, no LLM API calls). For each scene type, the pipeline produced a valid scene graph, passed validation, and rendered successfully in Blender EEVEE (64 samples, 1280×720 resolution). Figure ?? shows the rendered outputs.

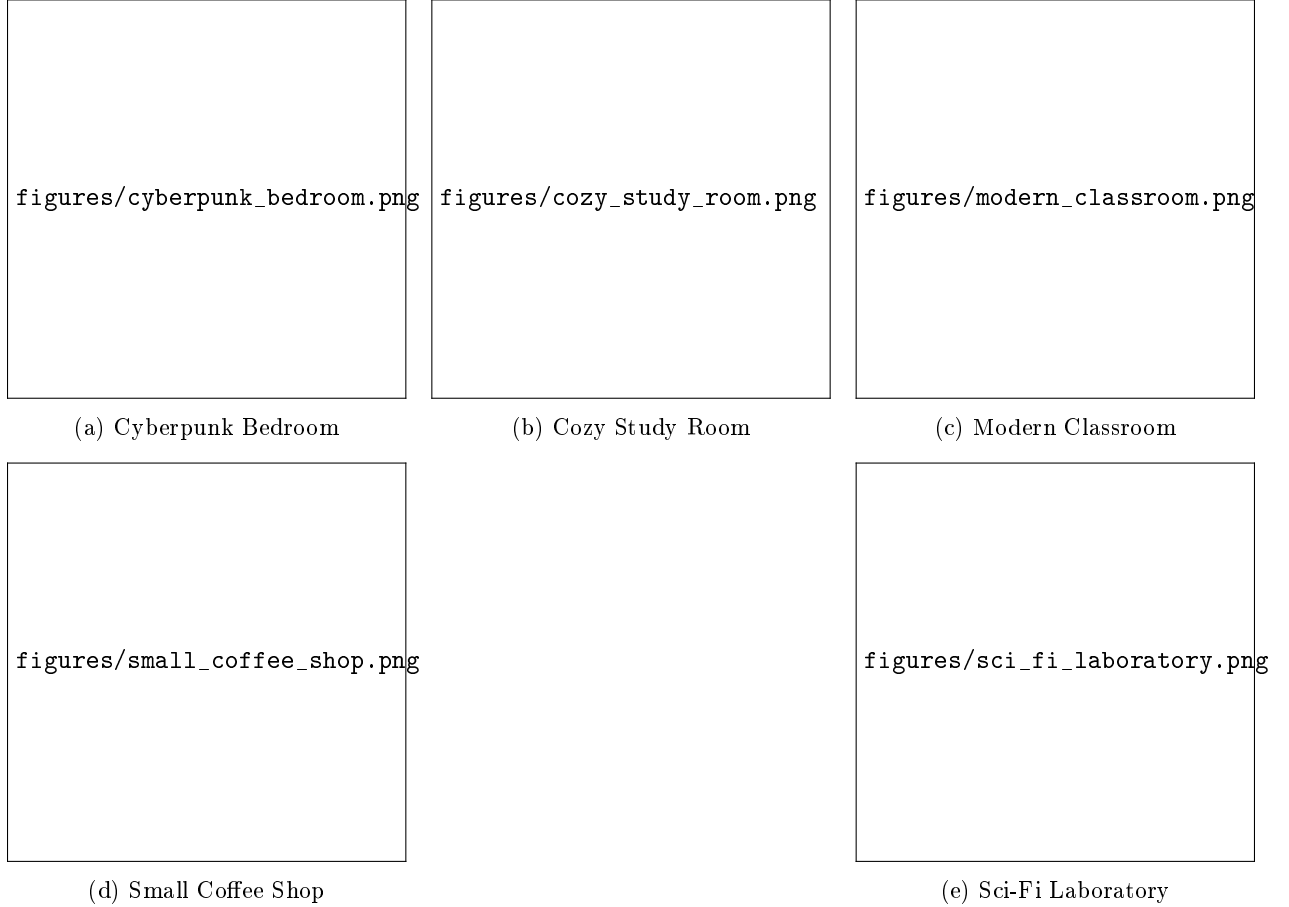


Figure 3: Rendered 3D scenes from natural language prompts across five scene types. All scenes were generated using mock-mode planning without external 3D assets or real LLM API calls. Rendered in Blender EEVEE at 1280×720 resolution.

4.2 Layout Pipeline Analysis

We conducted a detailed analysis of the 6-stage layout pipeline on the cyberpunk bedroom scene (6 objects: bed, desk, carpet, monitor, neon_light, poster, window). Table ?? shows the computed positions and categories.

4.3 Collision Detection and Resolution

The layout report recorded the following statistics:

- **Collisions before resolution:** 1 (bed_1 $\leftrightarrow$ carpet_1, overlap: $1.6 \times 0.95 \times 0.03$ m).
- **Collisions after resolution:** 0 (resolved by shifting carpet along $-y$ in 0.3 m steps).

Table 1: Per-object layout results for the cyberpunk bedroom scene. Floor objects have $z = \text{height}/2$ (sitting on floor); wall objects are attached to walls with thickness offset; the monitor sits on the desk via the `on` relation.

Object	Type	Category	Center X	Center Y	Center Z
bed_1	bed	floor	-1.62	1.25	0.275
desk_1	desk	floor	1.68	1.10	0.375
carpet_1	carpet	floor	0.68	-0.15	0.020
monitor_1	monitor	on_target	1.68	1.10	0.985
neon_1	neon_light	wall	0.00	2.395	1.650
poster_1	poster	wall	-2.52	0.00	1.650
window_1	window	wall	0.00	2.395	1.650

- **Out-of-bounds before clamping:** 3 (neon_1, poster_1, window_1 — these are wall objects, expected to be outside the interior since they attach to walls).
- **Out-of-bounds after clamping:** 0.

This demonstrates that the iterative shifting strategy effectively resolves overlaps between floor objects while maintaining room containment.

4.4 Test Coverage

The project includes 81 pure-Python unit tests organized into 6 test classes:

Table 2: Test suite breakdown. All tests run without Blender.

Test Class	Tests
TestSceneSchema	15
TestValidator	11
TestAABB	4
TestIntersection	4
TestRoomContainment	4
TestPlaceOnWall	5
TestLayoutPipeline	10
<i>Other (existing)</i>	<i>28</i>
Total	81

4.5 Ablation: Without Geometry-Safe Layout

To quantify the impact of the geometry-safe layout pipeline, we compared the baseline system (direct coordinate usage from scene graph) against our 6-stage pipeline. Without the pipeline:

- Floor objects frequently penetrate the floor plane (center at $z = 0$, so half the object is below the floor).
- Wall objects float at arbitrary depths rather than attaching to wall surfaces.
- `on` relations place objects at incorrect heights.
- Collisions between adjacent objects go undetected and unresolved.

Our pipeline fixes all four issues deterministically, without requiring any training or tuning.

5 Discussion

5.1 Limitations

- **Primitive-only geometry:** Objects are approximated by combinations of Blender primitives (cubes, cylinders, planes). Detailed meshes (e.g., ornate furniture) are not supported in the current version.
- **Mock planner:** In mock mode, scene graphs are selected from pre-written examples by keyword matching. The system does not generate truly novel scene compositions without an LLM backend.
- **No inter-object penetration resolution:** The current collision resolver only handles 2D floor-plane overlaps. Objects could theoretically occupy the same vertical space if placed by different relations.
- **Stage-set room only:** The room model assumes a rectangular box with an open front. Non-rectangular rooms and multi-room layouts are not supported.
- **Visual critic not integrated:** The visual feedback loop (render $\rightarrow$ critique $\rightarrow$ repair) is designed but not yet implemented with a real VLM.

5.2 Future Work

- **Real LLM integration:** Connect the Planner, Validator, and Repair agents to an OpenAI-compatible API (e.g., Google AI Studio, GLM) to enable true “zero-shot” scene generation from arbitrary prompts.
- **VLM-based visual critique:** Implement the planned render $\rightarrow$ VLM critique $\rightarrow$ automatic repair loop for iterative scene improvement.
- **Enhanced collision resolution:** Extend the collision resolver to 3D volumetric overlaps and support rotational adjustments.
- **External asset import:** Support importing .glb/.obj models to replace primitive approximations with high-quality meshes.
- **Interactive web interface:** Build a web frontend for real-time scene design and preview.
- **Physics simulation:** Integrate Blender’s physics engine for gravity-based placement verification.

6 Conclusion

We presented BlenderSceneAgent, a lightweight multi-agent system for 3D indoor scene generation from natural language descriptions. Our key innovations are: (1) a scene graph intermediate representation that decouples AI planning from Blender execution; (2) a 6-stage geometry-safe layout pipeline that deterministically computes physically valid object positions using AABB collision detection; and (3) a mock-first multi-agent architecture that enables rapid development and testing without external API dependencies.

The system supports 25 object types, 9 spatial relations, and 6 scene categories, all rendered using only Blender primitives. With 81 passing unit tests and a comprehensive layout debugging mode, the framework is robust and extensible. We believe the scene-graph-as-IR paradigm, combined with deterministic geometric reasoning, offers a practical and interpretable alternative to end-to-end neural 3D generation, particularly for educational and prototyping contexts.

Acknowledgments

I thank the course instructors for their guidance and the open-source community for the tools and libraries that made this project possible (Blender, Pydantic, pytest).

References

- Y. Wang *et al.*, “GenPilot: A Multi-Agent System for Test-Time Prompt Optimization in Image Generation,” in *Findings of EMNLP*, 2025.
- Paper2Poster Team, “Paper2Poster: Towards Multimodal Poster Automation from Scientific Papers,” in *NeurIPS Datasets and Benchmarks Track*, 2025.
- PosterForest Team, “PosterForest: Hierarchical Multi-Agent Collaboration for Scientific Poster Generation,” *arXiv:2508.21720*, 2025.
- H. Xia *et al.*, “SAGE: Scalable Agentic 3D Scene Generation for Embodied AI,” *arXiv:2602.10116*, 2026.
- S. Lu *et al.*, “LL3M: Large Language 3D Modelers,” *arXiv:2508.08228*, 2025.
- D. Mallis *et al.*, “CAD-Assistant: Tool-Augmented VLLMs as Generic CAD Task Solvers,” in *ICCV*, 2025.
- X. Liu *et al.*, “WorldCraft: Photo-Realistic 3D World Creation and Customization via LLM Agents,” *arXiv:2502.15601*, 2025.
- J. Chen *et al.*, “Idea23D: Collaborative LMM Agents Enable 3D Model Generation from Interleaved Multi-modal Inputs,” in *COLING*, 2025.
- B. Wen *et al.*, “3D Scene Generation: A Survey,” *arXiv:2505.05474*, 2025.
- Z. Yuan *et al.*, “Mora: Enabling Generalist Video Generation via a Multi-Agent Framework,” *arXiv:2403.13248*, 2024.
- H. Shi *et al.*, “AniMaker: Multi-Agent Animated Storytelling with MCTS-Driven Clip Generation,” in *SIG-GRAPH Asia*, 2025.
- J. Shi *et al.*, “PresentAgent: Multimodal Agent for Presentation Video Generation,” in *EMNLP System Demonstrations*, 2025.
- Blender Foundation, “Blender 5.0 Python API Documentation,” <https://docs.blender.org/api/current/>, 2026.
- Pydantic Team, “Pydantic v2: Data Validation Using Python Type Hints,” <https://docs.pydantic.dev/>, 2024.

Author Contributions

This is a solo project. All system design, implementation, testing, and documentation were completed by the author.

Name	Student ID	Contributions
	23300680108	All aspects: system architecture, multi-agent pipeline, Pydantic schema, layout pipeline, scene builder, Blender scripting, materials & lighting, testing framework, documentation, experimental evaluation